

732G57 Maskininlärning för statistiker

Föreläsning 4

Josef Wilzén

IDA, Linköping University, Sweden

- K-närmaste grannar
- One-Dimensional Kernel Smoothers
- Lokal Regression
- GAM: Generaliserade Additiva Modeller

Idé basera predikation på de K datapunkter som är närmast.

Ger en icke-parametrisk metod för klassificering och regression.

Problem: Vad är närmast?

Vi behöver något som talar om för oss hur nära två datapunkter är.
Finns många alternativ som man kan välja, som ger olika resultat.

Euklidiskt avstånd

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{k=1}^p (x_k - y_k)^2}$$

Manhattan avstånd

$$d(\mathbf{x}, \mathbf{y}) = \sum_{k=1}^p |x_k - y_k|$$

K-närmaste grannar - klassificering

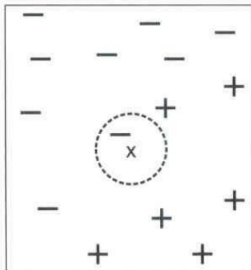
1. Låt k vara ditt valda antal grannar och D din träningsdata.
2. För varje testdata $z = (\mathbf{x}', y')$ $\in D$:
 - 2.1 Beräkna $d(\mathbf{x}, \mathbf{x}')$ (avståndet mellan z och all träningsdata)
 - 2.2 Välj $D_z \subseteq D$, de k närmaste träningsdatan till z
 - 2.3 Låt $y' = \arg \max_v \sum_{(\mathbf{x}_i, y_i) \in D_z} \mathbf{1}_{v=y_i}$

2.3 är majoritetsval. Kan också vikta detta värde med avståndet:

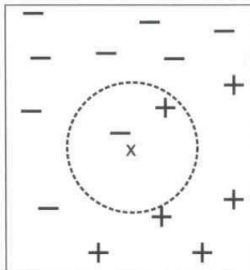
$$2.3 \quad y' = \arg \max_v \sum_{(\mathbf{x}_i, y_i) \in D_z} w_i \mathbf{1}_{v=y_i}.$$

För regression används medelvärde, alternativt viktat medelvärde.

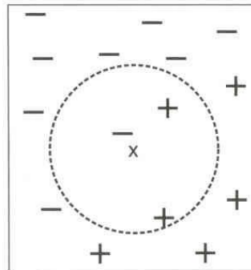
K-närmaste grannar



(a) 1-nearest neighbor



(b) 2-nearest neighbor



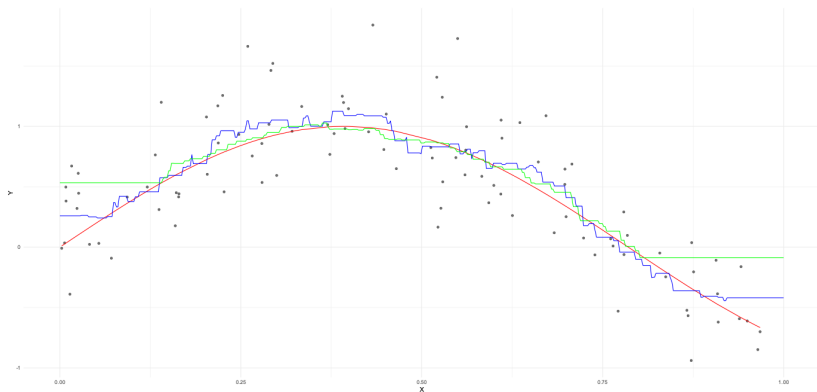
(c) 3-nearest neighbor

- Målet med modellen är att prediktera nya observationer
 - → enkel/lat metod
- Avståndsmått påverkar, påverkas stort av olika skalor på variabler
- Långsam anpassning.
- Känslig mot brus.
- Val av K har stor betydelse!
 - Litet K ger överanpassning.
 - Stort K ger underanpassning.
 - Korsvalidering kan användas för att bestämma K.
- Producerar godtyckligt utformade beslutsgränser.
- Problem i högre dimensioner → **Curse of dimensionality**

K-närmaste grannar: exempel

Regressionsdata: röda linjen + brus

$$\hat{g}(x) = \text{Medel}(y_i | x_i \in N_k(x)).$$



KNN: Blå har $K = 10$ och grön har $K = 30$.

One-Dimensional Kernel Smoothers

- KNN-regression ger samma vikt till alla punkter i grannskapet.
- Detta kan ge hopp och en ojämn skattad funktion.
- En lösning är att låta vikten bero på avståndet till x_0 .

Vi vill skatta $g(x_0)$ och använder ett viktat medelvärde:

$$\hat{g}(x_0) = \frac{\sum_{i=1}^n K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^n K_\lambda(x_0, x_i)}.$$

- Observationer nära x_0 får hög vikt.
- Observationer långt från x_0 får låg vikt.
- $K_\lambda(x_0, x_i)$ kallas en *kernel* eller viktfunktion.

One-Dimensional Kernel Smoothers

En kernel används för att ge större vikt till punkter nära x_0 och mindre vikt till punkter längre bort.

$$\hat{g}(x_0) = \frac{\sum_{i=1}^n K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^n K_\lambda(x_0, x_i)}$$

Vanliga kernels:

- Triangel
- Epanechnikov
- Quartic
- Gaussian

Bandwidth

Hyperparametern λ styr hur stort område kring x_0 som påverkar skattningen.

- Litet $\lambda \rightarrow$ låg bias, hög varians.
- Stort $\lambda \rightarrow$ hög bias, låg varians.
- Valet av λ är ofta viktigare än valet av kernel.

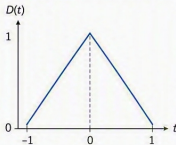
One-Dimensional Kernel Smoothers

Vanliga 1-dimensionella kernel-funktioner

En kernel skrivs som $K_\lambda(x_0, x) = \frac{1}{\lambda} D\left(\frac{x - x_0}{\lambda}\right)$, där $D(t)$ är en täthetsfunktion med $\int_{-\infty}^{\infty} D(t) dt = 1$.

Triangulär (Bartlett)

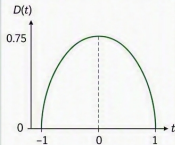
$$D(t) = (1 - |t|) \mathbf{1}(|t| \leq 1)$$



- Stöd: $|t| \leq 1$
- Kontinuerlig
- Linjära sidor

Epanechnikov

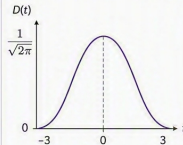
$$D(t) = \frac{3}{4} (1 - t^2) \mathbf{1}(|t| \leq 1)$$



- Stöd: $|t| \leq 1$
- Kontinuerlig
- Optimal i MSE-mening

Gaussisk (normal)

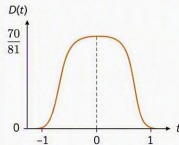
$$D(t) = \frac{1}{\sqrt{2\pi}} e^{-\frac{t^2}{2}}, -\infty < t < \infty$$



- Stöd: hela reella linjen
- Kontinuitet: oändligt många derivator
- Vikter avtar exponentiellt

Tri-kubisk

$$D(t) = \frac{70}{81} (1 - |t|^3)^3 \mathbf{1}(|t| \leq 1)$$



- Stöd: $|t| \leq 1$
- Kontinuerlig
- Mycket slät i kanterna



Bandbredd (λ)

Större $\lambda \Rightarrow$ bredare kernel (mer utjämning). Mindre $\lambda \Rightarrow$ smalare kernel (mindre utjämning).



Kärnans roll

Punkter nära x_0 får hög vikt, punkter långt bort får låg vikt.

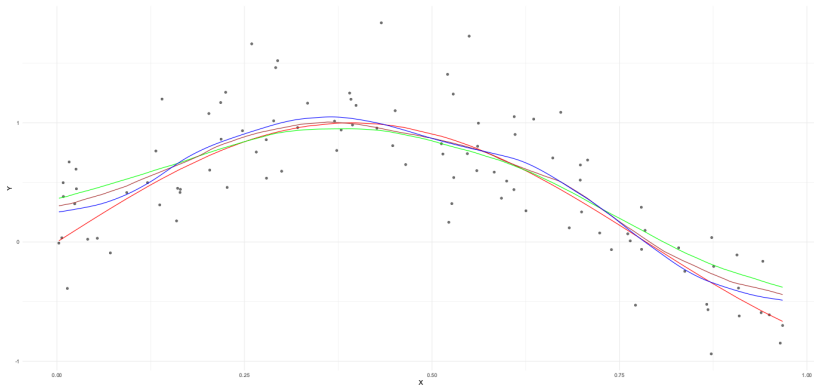


Praktiskt

Valet av λ har oftast större inverkan på resultatet än valet av kernel.

One-Dimensional Kernel Smoothers

Vi testar tre olika kernels (Triangel - Grön, Epanechnikov - Brun, Quartic - Blå) och får följande resultat



One-Dimensional Kernel Smoothers

- Ett sätt att göra "viktrat KNN" som ger en "mjuk" funktion.
- Vanligtvis sätter vi inte ett antal grannar.
 - Låter λ styra hur stort fönster vi kollar i.
- Många olika val av kernels.
- Valet av λ är ofta viktigare än valet av kernel.
- Problem vid kanterna.

Vi har sedan tidigare delat upp variabelrummet i olika delar och gjort regression på varje del.

Istället för att dela upp rummet i förväg och göra regression i varje del, varför inte välja ett intervall runt den punkt där vi vill beräkna regressionen?

Hur välja område?

Inspirerat av det One-Dimensional Kernel Smoothers

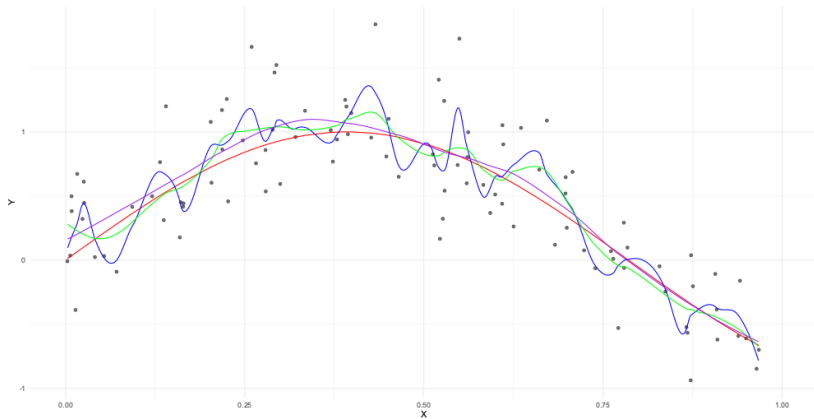
Gör (linjär) regression vid en punkt x_0 där vi viktar alla fel med en kernel.

För varje punkt x_0 där vi vill beräkna funktionen löser vi ett regressionsproblem

$$\min_{\beta(x_0)} \sum_{i=1}^n K_{\lambda}(x_0, x_i) [y_i - \beta_0(x_0) + \beta_1(x_0)x_i]^2.$$

Lokal Regression

Skattar lokal regression med olika λ och Tri-Cube



- Kan såklart göra polynomregression istället för linjär regression.
- Går att göra för multipel regression,
 - Vanligt att man bara gör lokal regression för några enskilda parametrar.
 - Kan göra simultan lokal regression där vi skattar ett p -dimensionellt plan.
 - Funkar (generellt) dåligt för p större än 4.

LOESS = **L**Ocal regr**ESS**ion

- Vid varje punkt anpassas en lokal regressionsmodell.
- Närliggande observationer får högre vikt än avlägsna observationer.
- Standard i `loess()` är lokal kvadratisk regression (`degree = 2`).

Anpassning i R:

```
fit <- loess(y ~ x, span = 0.5)
```

`geom_smooth()`: "stats::loess() is used for less than 1,000 observations..."

Parametern `span` styr storleken på det lokala grannskapet:

- `span = 0.2`: använder ungefär 20% av observationerna lokalt.
- `span = 0.8`: använder ungefär 80% av observationerna lokalt.
- Litet `span`: flexibel modell, risk för överanpassning.
- Stort `span`: slät modell, risk för underanpassning.

`span` i LOESS är ungefär motsvarigheten till bandwidth λ i kernel smoothing. Skillnaden är att λ mäts i x -enheter medan `span` anger hur stor andel av observationerna som används lokalt.

Ordo-notation beskriver hur beräkningstiden växer när antalet observationer n ökar.

Komplexitet	Tolkning
$O(1)$	Konstant tid
$O(\log n)$	Logaritmisk tillväxt
$O(n)$	Linjär tillväxt
$O(n^2)$	Kvadratisk tillväxt
$O(n^3)$	Kubisk tillväxt

Ju **långsammare** komplexiteten växer, desto **bättre skalar** algoritmen till stora datamängder.

Exempel på komplexitet

Antag att vi dubblar antalet observationer:

$$n \longrightarrow 2n.$$

Då förändras beräkningstiden ungefär enligt:

Komplexitet	Förändring
$O(1)$	$1\times$
$O(\log n)$	något större än $1\times$
$O(n)$	$2\times$
$O(n^2)$	$4\times$
$O(n^3)$	$8\times$

Exempel

Om antalet observationer fördubblas $n \rightarrow 2n$ så blir en algoritm med komplexitet $O(n^3)$ ungefär 8 gånger långsammare.

Beräkningskostnad – jämförelse

Antag att vi har n observationer.

Tabellen visar den ungefärliga kostnaden för att:

- Träna modellen,
- Beräkna modellens skattningar i samtliga n träningspunkter.
- Komplexiteterna är ungefärliga och avser moderna implementationer.

Metod	Träning	Prediktion i n punkter
Linjär regression (p variabler)	$O(np^2 + p^3)$	$O(np)$
Naturliga splines (K knutar)	$O(nK^2)$	$O(nK)$
Smoothing splines	$O(n)$	$O(n)$
KNN (k grannar)	$O(1)$	$O(n^2)$
Kernel smoothers	$O(1)$	$O(n^2)$
Lokal regression (LOESS)	$O(1)$	$O(n^2)$

- Om antalet knutar (K) hålls konstant blir detta ungefär $O(n)$.

- Linjär regression skapar en global modell.
- Naturliga splines skapar $K+1$ "lokala" modeller
- KNN, kernel smoothers och LOESS gör huvuddelen av arbetet vid prediktion.
- Smoothing splines kräver dyrare matrisberäkningar vid träning.

KNN, kernel smoothers och LOESS kallas ibland för *lazy learners* eftersom de gör relativt lite arbete vid träning och mer arbete vid prediktion.

Generaliserade Additiva Modeller

Det vi gjort denna och förra föreläsning handlar om att presentera flexibla modeller för att prediktera y givet x .

Nu ska vi använda detta för att skapa flexibla modeller för att prediktera y givet $x_1, x_2, x_3, \dots, x_p$.

Generaliserade Additiva Modeller (GAM) är en generell modell för att utöka linjär regression genom att tillåta icke-linjära funktioner av varje variabel, samtidigt som modellen är additiv.

Gemensamt för alla GAM modeller är att vi i vår regressions- eller klassificeringsmodell byter ut

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p,$$

till

$$\beta_0 + \beta_1 g_1(x_1) + \beta_2 g_2(x_2) + \dots + \beta_p g_p(x_p),$$

där g_i är en ("mjuk") icke-linjär funktion.

För regression får vi följande typ av modell,

$$y_i = \beta_0 + \beta_1 g_1(x_1) + \beta_2 g_2(x_2) + \dots + \beta_p g_p(x_p) + \varepsilon_i,$$

där vi är fria att välja varje g_i själva.

T.ex. kan vi välja att några ska vara splines, någon kanske är en steg-funktion och någon är identitetsfunktionen.

För klassificering får vi följande typ av modell,

$$\log \left(\frac{p(x)}{1 - p(x)} \right) = \beta_0 + \beta_1 g_1(x_1) + \beta_2 g_2(x_2) + \dots + \beta_p g_p(x_p) + \varepsilon_i,$$

där vi är fria att välja varje g_i själva.

Här kan vi igen aktivt välja vilka typer av funktioner som passar bäst per variabel.

Generaliserade Additiva Modeller

- GAMs låter oss anpassa en icke-linjär funktion för varje förklarande variabel.
- Eftersom modellen är additativ kan vi fortfarande undersöka effekten av varje förklarande variabel genom att hålla de andra konstanta.
- Ger en bra avvägning mellan flexibilitet och tolkningsbarhet
- Hur "mjuka" varje funktion är kan vi beskriva genom frihetsgraderna.
- Den största nackdelen är att modellen måste vara additativ.
 - Detta gör att interaktioner mellan variabler missas.
 - Vi kan lägga till specifika interaktioner, men detta måste göras manuellt.

Modellval för GAM

Modellval i GAM handlar huvudsakligen om att välja

- vilka variabler som ska ingå,
- vilka typer av funktioner som ska användas (linjär/icke-linjär)
- hur många frihetsgrader varje funktion ska få.

Vanliga metoder:

- Korsvalidering (Cross Validation)
- Generalized Cross Validation (GCV)
- AIC, BIC
- Penaliserade metoder med regularisering

Modellval i GAM handlar i stor utsträckning om att välja hur flexibla funktionerna $g_1, \dots, g_p$ ska vara.

`gamselect` (GAM Selection) använder regularisering för att automatiskt välja modellens komplexitet.

För varje variabel finns tre möjligheter:

Ej vald $\implies$ Linjär effekt $\implies$ Icke-linjär effekt

Modellval i `gamse1`

Regulariseringsparametern λ styr hur komplex modellen får vara.

- Stor λ :
 - Fler koefficienter krymps mot noll.
 - Färre variabler väljs.
 - Linjära effekter föredras framför icke-linjära.
- Liten λ :
 - Fler variabler inkluderas.
 - Mer flexibla funktioner tillåts.
 - Modellen blir mer komplex.

λ styr avvägningen mellan modellens komplexitet och dess anpassning till data.

Vanligtvis väljs λ med korsvalidering: `cv.gamse1(...)`